

LEARNING CHESS WITHOUT CHESS KNOWLEDGE: SUPERVISED BOOTSTRAPPING AND GATED SELF-PLAY REINFORCEMENT LEARNING

Gaurav Vyas (Roll Number: 23035010370)

Internship / Term Project Report
Trimester 9, Project 3

B.Sc. (Honours) Data Science and Artificial Intelligence
Indian Institute of Technology Guwahati, India
g.vyas@op.iitg.ac.in

ABSTRACT

This report documents a project on training a compact neural network to play chess without hand-coded chess knowledge, using the AlphaZero recipe: a residual policy-value network trained first by supervised imitation of human games and then improved by self-play guided by Monte Carlo Tree Search. A 760 717-parameter network was trained on 676 648 positions mined from the Lichess open database, reaching a policy cross-entropy of 2.06. The central finding is methodological rather than architectural. The evaluation procedure originally used scored games from White’s perspective while the network alternated colours, which rendered 159 logged evaluation points across two training runs statistically vacuous. Once corrected scoring was applied, direct head-to-head play revealed that 300 iterations of self-play had made the model *weaker* than its own starting point (2.5 to 5.5 over eight games) even as training loss fell monotonically. Two root causes were identified and fixed: policy targets formed from 40-visit search histograms are dominated by sampling noise, and a mirror data-augmentation step was generating physically impossible board states. A corrected pipeline adds acceptance gating, which makes regression structurally impossible.

Index Terms— Reinforcement Learning, Monte Carlo Tree Search, Deep Learning, Self-Play, Evaluation Methodology, Experimental Design

1. INTRODUCTION

Game playing has long served as a testbed for machine learning because the reward is unambiguous and data can be generated without human annotation. AlphaZero [1] showed that a single neural network, trained purely by self-play and guided by Monte Carlo Tree Search (MCTS), can reach superhuman strength in chess, shogi and Go without any domain-specific heuristics. The recipe is attractive from a data-science standpoint because it is a closed loop: the model generates its own training distribution, and improvements compound.

That closed loop is also its central risk. When a model manufactures its own labels, errors are not merely tolerated but amplified, and the usual supervised-learning safety net — a held-out set whose labels are known to be correct — does not exist. The quantity a practitioner actually cares about, playing strength, is not the quantity being minimised, and the two can move in opposite directions without any warning appearing in the loss curve.

This project reproduces the AlphaZero recipe at a scale that fits

on a single desktop machine, and then subjects it to the measurement discipline that self-play demands. Section 2 states the problem and objectives. Section 3 describes the data pipeline, the representation, the network and the two training stages. Section 4 presents the experimental results, including a defect in the evaluation methodology that invalidated an entire training run. Section 5 interprets these findings, and Section 6 concludes.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1. Problem Statement

Given only the rules of chess and a corpus of human games, train a neural network with fewer than one million parameters to select strong moves, subject to the constraint that all data generation, training and evaluation run on a single consumer machine (six CPU cores, one mid-range GPU) within a few hours per experiment. The network receives no evaluation function, no opening book, no endgame tablebase and no hand-engineered features; it must learn the value of a position and the plausibility of a move entirely from data.

The scale constraint is not incidental. AlphaZero used 5000 TPUs and 800 MCTS simulations per move; this project has roughly six orders of magnitude less compute. The interesting question is therefore not whether the recipe reaches grandmaster strength, but *which parts of it survive aggressive downscaling* — and how one would know.

2.2. Objectives

1. To design and implement an end-to-end pipeline: data acquisition, board representation, network, search, training and evaluation.
2. To bootstrap the network by supervised imitation of human play, and quantify the resulting move-prediction quality.
3. To improve the bootstrapped network by self-play reinforcement learning, and *verify* that improvement with a metric valid under self-play conditions.
4. To diagnose any failure to improve, and to make the training procedure robust against it.

Objective 3 is deliberately phrased to separate *running* the algorithm from *demonstrating* that it worked. That separation turned out to carry the entire result.

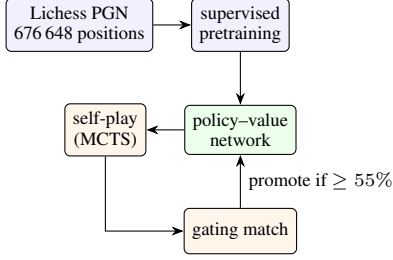


Fig. 1. The two-stage pipeline. Supervised pretraining supplies the initial policy; self-play then proposes updates, which are accepted only if they win a head-to-head match against the incumbent weights.

3. METHODOLOGY

3.1. Overall Workflow

Training proceeds in two stages (Figure 1). Stage 1 fits the network to human moves, giving it an immediate grasp of openings and basic tactics. Stage 2 replaces the human teacher with the network’s own search. An acceptance gate, added after the diagnosis in Section 4.4, sits between the two.

3.2. Data Pipeline

Data come from the Lichess open database [2], distributed as Zstandard-compressed PGN. Monthly dumps are streamed and decompressed incrementally rather than materialised, since a single month exceeds available memory when expanded.

Quality filtering is applied at the game level: only games in which *both* players are rated at least 1750 Elo are retained, so that the network imitates competent rather than average play. This filter is aggressive — it admits roughly 12% of games — and it is the main reason the final corpus is 676 648 positions drawn from approximately 9600 games rather than the 80 000 games read.

Each retained position yields one training example: an $18 \times 8 \times 8$ tensor, the index of the move the human actually played, and the game outcome expressed from the side to move. A subset of 40 000 examples is frozen as a *rehearsal anchor* (Section 3.6), and 6000 positions taken five to eight plies into games are retained as a self-play opening curriculum.

3.3. Representation and Action Encoding

The board is encoded as 18 binary planes of 8×8 : twelve piece-placement planes (six piece types $\times$ two colours), four castling-rights planes, one en-passant plane and one side-to-move plane.

The action space follows the Leela Chess Zero scheme [3]: each of the 64 origin squares carries 73 move planes — 56 queen-style moves (eight directions $\times$ seven distances), eight knight moves and nine underpromotions — giving $64 \times 73 = 4672$ actions, indexed as $\text{index} = \text{from} \times 73 + \text{plane}$. A round-trip self-test verifies that every legal move in a sample of positions encodes and decodes without collision.

3.4. Network Architecture

The network is a residual convolutional trunk [4] with two heads: a 3×3 convolution mapping 18 input planes to 64 channels, ten residual blocks of two 3×3 convolutions with batch normalisation [5], a

policy head (1×1 convolution to 73 planes, reshaped to 4672 logits) and a value head (1×1 convolution to one channel, a fully connected layer to 64 units, and a scalar through tanh). Total trainable parameters: 760 717.

The two heads share the trunk, so the model is trained multi-task: the value head learns *who is winning* while the policy head learns *which moves are plausible*, and the shared representation must serve both.

3.5. Stage 1: Supervised Pretraining

Given a position s , the human move m and the outcome $z \in \{-1, 0, +1\}$, the loss is

$$\mathcal{L}_{\text{sup}} = -\log p_{\theta}(m | s) + (v_{\theta}(s) - z)^2, \quad (1)$$

optimised with Adam [6], a cosine learning-rate schedule from 10^{-3} to 10^{-4} , and batch size 256 for three epochs.

3.6. Stage 2: Self-Play Reinforcement Learning

The network plays itself. Each move is selected by MCTS [7, 8], which descends the tree by the PUCT rule [9]

$$U(s, a) = Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)}, \quad (2)$$

where Q is the mean value of the child, P the network prior and N the visit count. Leaves are evaluated by the network; terminal positions use the true result. Dirichlet noise is added at the root during self-play only.

The normalised visit distribution π becomes the policy target and the game result z the value target, giving the AlphaZero loss

$$\mathcal{L} = \mathbb{E}_s \left[-\pi^{\top} \log p_{\theta}(s) + (v_{\theta}(s) - z)^2 \right] + \lambda \|\theta\|^2. \quad (3)$$

Two additions guard against catastrophic forgetting [10] of the supervised knowledge; the first is rehearsal on a frozen exemplar set [11]: 30% of every minibatch is drawn from the frozen rehearsal anchor, and self-play games start from curriculum positions 85% of the time.

Self-play runs in six CPU worker processes while gradient steps run on the GPU, overlapped so that neither device idles; self-play therefore uses weights one iteration stale, a standard relaxation.

3.7. Evaluation Methodology

Because self-play provides no held-out labels, playing strength must be measured by playing games. Three properties are required for such a measurement to carry information:

1. **Correct attribution.** A match result must be scored from the *network’s* perspective. If the network alternates colours and results are tallied as raw PGN strings, every win as Black is recorded as a loss.
2. **Independent games.** Two deterministic players replay one identical game. An N -game match between deterministic agents contains one game of information unless openings are randomised.
3. **An informative opponent.** A fixed, frozen reference opponent measures progress; an opponent that improves alongside the model measures nothing.

All three were violated in the original implementation. Their restoration is what makes Section 4 interpretable.

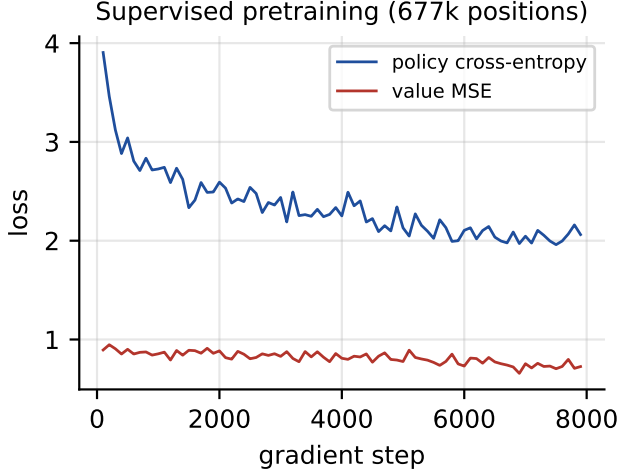


Fig. 2. Supervised stage. Policy cross-entropy converges to ≈ 2.06 ; value MSE plateaus near 0.73, reflecting the intrinsic difficulty of predicting a game outcome from a single mid-game position.

4. EXPERIMENTS AND RESULTS

4.1. Experimental Setup

All experiments ran on an Intel i5-12400F (six cores, twelve threads) with an NVIDIA RTX 3060, in PyTorch 2.13 [12] with `python-chess` [13] for rules. Self-play is CPU-bound (single-threaded Python MCTS); the GPU serves only batched gradient steps, so low GPU utilisation during training is expected rather than symptomatic. Measured throughput at twelve games per iteration on six workers is 42 s at 40 simulations and 180 s at 128.

4.2. Supervised Pretraining

Policy cross-entropy fell from 3.90 to 2.06 over three epochs (Figure 2). A cross-entropy of 2.06 over 4672 actions corresponds to assigning roughly 13% probability to the move a strong human actually chose, against a uniform-over-legal-moves baseline near 3.4.

Behavioural probes confirm the loss reflects real chess competence: from the initial position the network’s raw policy assigns 0.594 to $e4$ and 0.313 to $d4$; it answers $e4$ with $e5$ or $c5$; it captures a hanging queen; and it finds a mate in one. Against a uniformly random opponent it scores 85% over ten games.

4.3. A Defect in the Evaluation Metric

The evaluation harness alternated the network between colours but scored wins-as-White plus half the draws. Combined with the absence of opening randomisation, the consequence is a tally that can only be symmetric, and a score pinned near 50% regardless of the model:

```
[iter 454] eval:  {1-0:  0, 0-1:  0, 1/2-1/2: 10} (score
                  50%)
[iter 499] eval:  {1-0:  0, 0-1:  0, 1/2-1/2: 10} (score
                  50%)
```

159 evaluation points across two runs conveyed no information. The same defect reported the pretrained model at 44% against a random opponent when its true score is 85%.

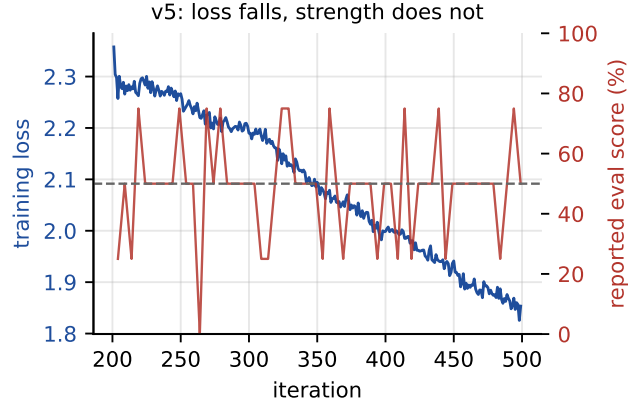


Fig. 3. The v5 run. Training loss (left axis) falls throughout, while the reported evaluation score (right axis) is pinned at 50% by the defect of Section 4.3. Direct head-to-head play shows the model in fact got weaker.

4.4. Self-Play Regressed

With corrected scoring and randomised openings, the final model of the 300-iteration self-play run was played directly against its own starting point over eight games:

v5 iter_500 2.5 – 5.5 pretrained iter_200

Self-play made the model *worse*. Throughout those 300 iterations the training loss fell monotonically from 2.36 to 1.85 (Figure 3). Loss is not a strength signal.

An earlier run without the rehearsal anchor failed more visibly: its policy degenerated to opening with $f3$, its value head returned -0.270 for the balanced initial position, and it no longer captured a hanging queen — textbook catastrophic forgetting [10]. Rehearsal prevented that collapse, but preventing collapse is not improvement.

4.5. Diagnosis: Search Quality Versus Target Quality

A natural hypothesis is that search is too weak to teach the network. It was tested by playing MCTS against the same network’s raw policy head at varying simulation counts (Figure 4). Search wins comfortably — 79% at only 40 simulations — so the hypothesis is false.

The distinction matters. Equation (3) trains on the full visit distribution π , not on its argmax. With 40 simulations spread over roughly 30 legal moves, π is a histogram of about 1.3 visits per move: its mode is informative, its shape is sampling noise. Fitting a sharp supervised policy to that noise flattens it.

A second, independent defect compounded this. A mirror augmentation flipped board files on half of every minibatch but did not exchange the kingside and queenside castling planes. After a file flip the king stands on $d1$ while the kingside-castling plane remains set — a position that cannot arise in chess, paired with features contradicting the pieces. Chess is not mirror-symmetric, precisely because castling and pawn direction break the symmetry; roughly half of every batch, including the rehearsal slice, was corrupted.

4.6. Corrected Pipeline

Four changes were made: castling-aware augmentation (positions retaining castling rights are not mirrored), 128 simulations instead

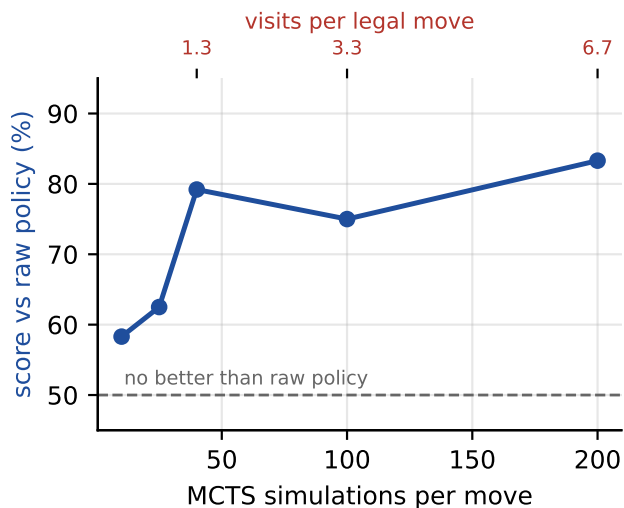


Fig. 4. MCTS against the network’s own greedy policy. Search is clearly stronger than the raw policy even at 40 simulations, so the *argmax* of the search is a good move; the upper axis shows why the *distribution* is nonetheless a poor target.

Table 1. Corrected run: outcome summary.

Quantity	Value
Iterations completed	[PENDING]
Gate matches / promotions	[PENDING]
Final score vs. frozen baseline	[PENDING]
Final score vs. random opponent	[PENDING]

of 40 (≈ 4.3 visits per legal move), 80 gradient steps per iteration instead of 200, and a reduced learning rate.

The decisive change is *acceptance gating*, following AlphaGo Zero [14]: self-play always generates from the best weights so far, and updated weights are promoted only after scoring at least 55% in a head-to-head match against the incumbent. Under gating the deployed model cannot get worse; the worst case is that nothing is promoted.

Under corrected scoring the model measures 90% against a random opponent, against the 50% the defective metric had reported throughout. Final outcomes of the corrected run are summarised in Table 1.

5. DISCUSSION

Optimisation success is not task success. The single most transferable lesson is that a monotonically decreasing loss curve coexisted with a model getting steadily worse at the task. The loss in Equation (3) is computed against targets the model itself produced; minimising it measures agreement with those targets, not chess strength. Any self-supervised or self-training pipeline shares this structure, and therefore shares this failure mode.

A broken metric is worse than no metric. The evaluation defect was costly not because it was subtle but because it was *reassuring*. It reported 50%, a plausible-looking number, for 159 consecutive measurements spanning two multi-hour runs. A metric that returns

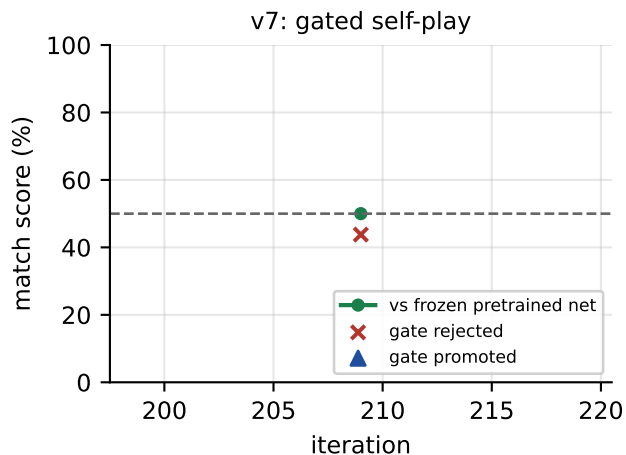


Fig. 5. The corrected run. Each gate point plots the candidate’s score against the incumbent (markers) and against the frozen pretrained network (line). The dashed line is equal strength.

a plausible constant is more dangerous than one that crashes. The symmetric tallies were visible in every log line and were read as evenly matched play rather than as the signature of a bug.

Sample efficiency governs downscaling. The failure was not a shortage of compute in aggregate but its allocation. Reducing simulations per move from 800 to 40 preserves the *argmax* of the search while destroying the calibration of its distribution, and Equation (3) consumes the distribution. When downscaling an algorithm, the question is not what a parameter does at full scale but which statistic of it the loss function actually reads.

Prefer structural guarantees to monitoring. Gating is valuable less because it improves the model than because it converts a silent failure into a visible one. A rejected gate is an unambiguous, immediate statement that the last ten iterations produced nothing. This is a general design preference: make the undesirable outcome impossible rather than merely detectable.

Limitations. Gate matches of eight to twelve games have wide confidence intervals; a 55% threshold on eight games is a weak test, and a longer match or a sequential test would be more reliable. The value head plateaus at an MSE of 0.73, indicating limited outcome-prediction skill from a single position. Finally, absolute strength is not calibrated against a rated engine, so results are stated as relative comparisons only.

6. CONCLUSION AND FUTURE WORK

A complete AlphaZero-style pipeline was implemented and trained on a single desktop machine. Supervised pretraining on 676 648 Elo-filtered Lichess positions produced a 760 717-parameter network that plays principled openings, wins material and finds simple mates, scoring 85% against a random opponent.

The project’s substantive contribution is negative and methodological. The self-play stage, as originally implemented, degraded the model, and the evaluation procedure in use was incapable of detecting this. Correcting the metric exposed the regression; controlled experiments isolated two causes, one statistical (visit-count histograms too sparse to serve as distributional targets) and one representational (an augmentation producing illegal board states); and

acceptance gating now makes regression structurally impossible.

Future work follows directly from the diagnosis. Batched leaf evaluation and subtree reuse would raise simulations per move several-fold at equal wall-clock cost, attacking the root cause rather than mitigating it. Sequential-probability-ratio testing [15] would make gate decisions at a stated confidence level instead of a fixed threshold on a short match. Calibrating against a rated engine such as Stockfish at reduced strength would convert relative comparisons into an Elo estimate [16].

7. ARTIFACTS AND DEMONSTRATIONS

- **Code Repository:** <https://github.com/realgauravyas/chess-ai> — full source for the network, search, both training stages, the dashboard, and the measurement scripts under `experiments/` that reproduce every number in this report. `RESULTS.md` records the measured findings and the defects fixed.
- **Interactive Dashboard:** `dashboard/server.py` serves a browser interface for playing any checkpoint, inspecting the search’s principal variation and move probabilities, and watching training curves live.
- **Video Demonstration:** [URL TO BE ADDED]

The presentation follows standard guidance on scientific writing [17]. AI-assisted tools were used for code review, debugging and drafting. All experimental results reported here were produced by the author’s own runs and are reproducible from the linked repository.

8. REFERENCES

- [1] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis, “A general reinforcement learning algorithm that masters chess, shogi, and go through self-play,” *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.
- [2] Lichess, “The lichess open database of standard rated games,” <https://database.lichess.org>, 2013, Accessed 2026.
- [3] Leela Chess Zero Team, “Leela chess zero: Open-source neural network chess engine,” <https://lczero.org>, 2018, Accessed 2026.
- [4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [5] Sergey Ioffe and Christian Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. Intl. Conf. on Machine Learning (ICML)*, 2015, pp. 448–456.
- [6] Diederik P. Kingma and Jimmy Ba, “Adam: A method for stochastic optimization,” in *Proc. Intl. Conf. on Learning Representations (ICLR)*, 2015.
- [7] Levente Kocsis and Csaba Szepesvári, “Bandit based Monte-Carlo planning,” in *Proc. European Conf. on Machine Learning (ECML)*, 2006, pp. 282–293.
- [8] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton, “A survey of Monte Carlo tree search methods,” *IEEE Trans. on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012.
- [9] Christopher D. Rosin, “Multi-armed bandits with episode context,” *Annals of Mathematics and Artificial Intelligence*, vol. 61, no. 3, pp. 203–230, 2011.
- [10] Michael McCloskey and Neal J. Cohen, “Catastrophic interference in connectionist networks: The sequential learning problem,” *Psychology of Learning and Motivation*, vol. 24, pp. 109–165, 1989.
- [11] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H. Lampert, “iCaRL: Incremental classifier and representation learning,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2001–2010.
- [12] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019, pp. 8024–8035.
- [13] Niklas Fiekas, “python-chess: A chess library for Python,” <https://python-chess.readthedocs.io>, 2024, Accessed 2026.
- [14] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis, “Mastering the game of Go without human knowledge,” *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.
- [15] Abraham Wald, “Sequential tests of statistical hypotheses,” *Annals of Mathematical Statistics*, vol. 16, no. 2, pp. 117–186, 1945.
- [16] Arpad E. Elo, “The rating of chessplayers, past and present,” 1978.
- [17] George D. Gopen and Judith A. Swan, “The science of scientific writing,” *American Scientist*, vol. 78, no. 6, pp. 550–558, 1990.